Project F

RISC-V Assembler Cheat Sheet

Published 14 Jun 2024 · Updated 03 Oct 2024

This cheat sheet provides a handy reference to 32-bit RISC-V instructions, registers, and concepts. Aimed at software developers, it groups instructions by purpose and includes common pseudoinstructions. Clicking on a *Guide* link takes you to the relevant section of the Project F RISC-V assembler guide for instruction explanation and examples.

Arithmetic | Bitwise Logic | Shift | Load Immediate | Load and Store | Jump and Function | Branch | Set | Counters | Misc Instructions | Instruction Terminology | RV32 ABI Registers | RISC-V Concepts

Instructions are from the base integer instruction set (RV32I) unless otherwise noted.

Arithmetic

Instr	Description	Use	Result	Guide
add	Add	`add rd, rs1, rs2`	`rd = rs1 + rs2`	arithmetic
addi	Add Immediate	`addi rd, rs1, imm`	`rd = rs1 + imm`	arithmetic
neg	Negate (p)	`neg rd, rs2`	`rd = -rs2`	arithmetic
sub	Subtract	`sub rd, rs1, rs2`	`rd = rs1 - rs2`	arithmetic

Instr	Description	Use	Result	Guide
mul	Multiply	`mul rd, rs1, rs2`	`rd = (rs1 * rs2) [31:0]`	multiply
mulh	Multiply High	`mulh rd, rs1, rs2`	`rd = (rs1 * rs2) [63:32]`	multiply
mulhu	Multiply High Unsigned	`mulhu rd, rs1, rs2`	`rd = (rs1 * rs2) [63:32]`	multiply
mulhsu	Multiply High Signed Unsigned	`mulhsu rd, rs1, rs2`	`rd = (rs1 * rs2) [63:32]`	multiply
div	Divide	`div rd, rs1, rs2`	`rd = rs1 / rs2`	divide
rem	Remainder	`rem rd, rs1, rs2`	`rd = rs1 % rs2`	divide

*Use **addi** for subtract immediate too. Multiply and divide instructions require the **M** extension.*

Bitwise Logic

Instr	Description	Use	Result	Guide
and	AND	`and rd, rs1, rs2`	`rd = rs1 & rs2`	logical
andi	AND Immediate	`andi rd, rs1, imm`	`rd = rs1 & imm`	logical
not	NOT (p)	`not rd, rs1`	`rd = ~rs1`	logical
or	OR	`or rd, rs1, rs2`	`rd = rs1 | rs2`	logical
ori	OR Immediate	`ori rd, rs1, imm`	`rd = rs1 | imm`	logical

Instr	Description	Use	Result	Guide
xor	XOR	`xor rd, rs1, rs2`	`rd = rs1 ^ rs2`	logical
xori	XOR Immediate	`xori rd, rs1, imm`	`rd = rs1 ^ imm`	logical

Shift

Instr	Description	Use	Result	Guide
sll	Shift Left Logical	`sll rd, rs1, rs2`	`rd = rs1 << rs2`	shift
slli	Shift Left Logical Immediate	`slli rd, rs1, imm`	`rd = rs1 << imm`	shift
srl	Shift Right Logical	`srl rd, rs1, rs2`	`rd = rs1 >> rs2`	shift
srli	Shift Right Logical Immediate	`srli rd, rs1, imm`	`rd = rs1 >> imm`	shift
sra	Shift Right Arithmetic	`sra rd, rs1, rs2`	`rd = rs1 >>> rs2`	shift
srai	Shift Right Arithmetic Immediate	`srai rd, rs1, imm`	`rd = rs1 >>> imm`	shift

Load Immediate

Instr	Description	Use	Result	Guide
li	Load Immediate (p)	`li rd, imm`	`rd = imm`	arithmetic
lui	Load Upper Immediate	`lui rd, imm`	`rd = imm << 12`	arithmetic

Instr	Description	Use	Result	Guide
auipc	Add Upper Immediate to PC	`auipc rd, imm`	`rd = pc + (imm << 12)`	branch

Load and Store

Instr	Description	Use	Result	Guide
lw	Load Word	`lw rd, imm(rs1)`	`rd = mem[rs1+imm]`	load
lh	Load Half	`lh rd, imm(rs1)`	`rd = mem[rs1+imm][0:15]`	load
lhu	Load Half Unsigned	`lhu rd, imm(rs1)`	`rd = mem[rs1+imm][0:15]`	load
lb	Load Byte	`lb rd, imm(rs1)`	`rd = mem[rs1+imm][0:7]`	load
lbu	Load Byte Unsigned	`lbu rd, imm(rs1)`	`rd = mem[rs1+imm][0:7]`	load
la	Load Symbol Address (p)	`la rd, symbol`	`rd = &symbol`	load
sw	Store Word	`sw rs2, imm(rs1)`	`mem[rs1+imm] = rs2`	store
sh	Store Half	`sh rs2, imm(rs1)`	`mem[rs1+imm][0:15] = rs2`	store
sb	Store Byte	`sb rs2, imm(rs1)`	`mem[rs1+imm][0:7] = rs2`	store

Jump and Function

Instr	Description	Use	Result	Guide
j	Jump (p)	**`j imm`**	**`pc += imm`**	jump
jal	Jump and Link	**`jal rd, imm`**	**`rd = pc+4; pc += imm`**	jump
jalr	Jump and Link Register	**`jalr rd, rs1, imm`**	**`rd = pc+4; pc = rs1+imm`**	jump
call	Call Function (p)	**`call symbol`**	**`ra = pc+4; pc = &symbol`**	function
ret	Return from Function (p)	**`ret`**	**`pc = ra`**	function

*You can use a label in place of a jump immediate, for example: **`j label_name`***

Branch

This page lists all branch instructions but you may prefer the branch instruction summary table.

Instr	Description	Use	Result	Guide
beq	Branch Equal	**`beq rs1, rs2, imm`**	**`if(rs1 == rs2) pc += imm`**	branch
beqz	Branch Equal Zero (p)	**`beqz rs1, imm`**	**`if(rs1 == 0) pc += imm`**	branch
bne	Branch Not Equal	**`bne rs1, rs2, imm`**	**`if(rs1 ≠ rs2) pc += imm`**	branch
bnez	Branch Not Equal Zero (p)	**`bnez rs1, rs2, imm`**	**`if(rs1 ≠ 0) pc += imm`**	branch

Instr	Description	Use	Result	Guide
blt	Branch Less Than	**blt rs1, rs2, imm**	**if(rs1 < rs2) pc += imm**	branch
bltu	Branch Less Than Unsigned	**bltu rs1, rs2, imm**	**if(rs1 < rs2) pc += imm**	branch
bltz	Branch Less Than Zero (p)	**bltz rs1, imm**	**if(rs1 < 0) pc += imm**	branch
bgt	Branch Greater Than (p)	**bgt rs1, rs2, imm**	**if(rs1 > rs2) pc += imm**	branch
bgtu	Branch Greater Than Unsigned (p)	**bgtu rs1, rs2, imm**	**if(rs1 > rs2) pc += imm**	branch
bgtz	Branch Greater Than Zero (p)	**bgtz rs1, imm**	**if(rs1 > 0) pc += imm**	branch
ble	Branch Less or Equal (p)	**ble rs1, rs2, imm**	**if(rs1 ≤ rs2) pc += imm**	branch
bleu	Branch Less or Equal Unsigned (p)	**bleu rs1, rs2, imm**	**if(rs1 ≤ rs2) pc += imm**	branch
blez	Branch Less or Equal Zero (p)	**blez rs1, imm**	**if(rs1 ≤ 0) pc += imm**	branch
bge	Branch Greater or Equal	**bge rs1, rs2, imm**	**if(rs1 ≥ rs2) pc += imm**	branch
bgeu	Branch Greater or Equal Unsigned	**bgeu rs1, rs2, imm**	**if(rs1 ≥ rs2) pc += imm**	branch
bgez	Branch Greater or Equal Zero (p)	**bgez rs1, imm**	**if(rs1 ≥ 0) pc += imm**	branch

You can use a label in place of a branch immediate, for example:

beq t0, t1, label_name

Set

Instr	Description	Use	Result	Guide
slt	Set Less Than	**slt rd, rs1, rs2**	**rd = (rs1 < rs2)**	set
slti	Set Less Than Immediate	**slti rd, rs1, imm**	**rd = (rs1 < imm)**	set
sltu	Set Less Than Unsigned	**sltu rd, rs1, rs2**	**rd = (rs1 < rs2)**	set
sltiu	Set Less Than Immediate Unsigned	**sltui rd, rs1, imm**	**rd = (rs1 < imm)**	set
seqz	Set Equal Zero (p)	**seqz rd, rs1**	**rd = (rs1 == 0)**	set
snez	Set Not Equal Zero (p)	**snez rd, rs1**	**rd = (rs1 ≠ 0)**	set
sltz	Set Less Than Zero (p)	**sltz rd, rs1**	**rd = (rs < 0)**	set
sgtz	Set Greater Than Zero (p)	**sgtz rd, rs1**	**rd = (rs1 > 0)**	set

Counters

Instr	Description	Use	Result	Guide
rdcycle	CPU Cycle Count (p)	**rdcycle rd**	**rd = csr_cycle[31:0]**	not yet avail

Instr	Description	Use	Result	Guide
rdcycleh	CPU Cycle Count High (p)	`rdcycleh rd`	`rd = csr_cycle[63:32]`	not yet avail
rdtime	Current Time (p)	`rdtime rd`	`rd = csr_time[31:0]`	not yet avail
rdtimeh	Current Time High (p)	`rdtimeh rd`	`rd = csr_time[63:32]`	not yet avail
rdinstret	CPU Instructions Retired (p)	`rdinstret rd`	`rd = csr_instret[31:0]`	not yet avail
rdinstreth	CPU Instructions Retired High (p)	`rdinstreth rd`	`rd = csr_instret[63:32]`	not yet avail

*The counter instructions require the **Zicntr** and **Zicsr** extensions but were originally part of the base instruction set.*

Misc Instructions

Instr	Description	Use	Result	Guide
ebreak	Environment Break (Debugger Call)	`ebreak`	–	not yet avail
ecall	Environment Call (OS Function)	`ecall`	–	not yet avail
fence	I/O Ordering	`fence`	–	not yet avail
mv	Copy Register (p)	`mv rd, rs1`	`rd = rs1`	arithmetic
nop	No Operation (p)	`nop`	–	arithmetic

*The **fence** instruction requires the **Zifencei** extension but was originally part of*

the base instruction set.

Instruction Terminology

- **imm** - immediate value (normally sign extended)
- **mem** - memory
- **(p)** - pseudoinstruction
- **pc** - program counter
- **pc+4** - next instruction on RV32
- **ra** - return address register (**x1**)
- **rd** - destination register
- **rs1** - first source register
- **rs2** - second source register
- **symbol** - symbol (may be label in asm)

RV32 ABI Registers

ABI Name	Register	Description	Preserved
zero	x0	always 0 (zero)	n/a
ra	x1	return address	no
sp	x2	stack pointer	yes
gp	x3	global pointer*	n/a
tp	x4	thread pointer*	n/a

ABI Name	Register	Description	Preserved
t0	x5	temporary	no
t1	x6	temporary	no
t2	x7	temporary	no
fp (s0)	x8	frame pointer†	yes
s1	x9	saved register	yes
a0	x10	function argument‡	no
a1	x11	function argument‡	no
a2	x12	function argument	no
a3	x13	function argument	no
a4	x14	function argument	no
a5	x15	function argument	no
a6	x16	function argument	no
a7	x17	function argument	no
s2	x18	saved register	yes
s3	x19	saved register	yes
s4	x20	saved register	yes
s5	x21	saved register	yes
s6	x22	saved register	yes
s7	x23	saved register	yes
s8	x24	saved register	yes

ABI Name	Register	Description	Preserved
s9	x25	saved register	yes
s10	x26	saved register	yes
s11	x27	saved register	yes
t3	x28	temporary	no
t4	x29	temporary	no
t5	x30	temporary	no
t6	x31	temporary	no

*Let the compiler/linker use the global **gp** and thread **tp** pointers; ignore them in your own code.
†The frame pointer **fp** supports local variables but can be used as a regular saved register.
‡Argument registers **a0** and **a1** also handle the function return value.

RISC-V Concepts

Important RISC-V concepts, briefly explained.

- Data Sizes (word, half, byte)
- Extensions (customising RISC-V cores)
- Load-Store Architecture
- Preserved Registers (function calls)
- Pseudoinstructions

- Sign Extension
- Stack
- Zero Register

What's Next?

Check out Compiler Explorer for RISC-V.

References

- RISC-V Technical Specifications (riscv.org)

asm riscv

Project F: FPGA and RISC-V. Only hardware makes it possible! © 2025 Will Green.